# How `gpedm_py` works

Mathematical description of the Python port of GPEDM
used in the IceLab-EDM RespiCast forecast

Describes `gpedm_py.py`, a port of GPEDM 0.0.0.9010, and the weekly forecast built on it, as of 14 October 2026

## 1 Overview

`gpedm_py.py` fits a Gaussian process to a delay-embedded time series and predicts from it. It ports, function by function, the three functions of the R package GPEDM [1] that the forecast calls: `makelags`, `fitGP` and `predict`. The method is Gaussian-process empirical dynamic modelling [2], with the variable-step lags of [3] for series that have gaps.

The calculation has four steps (Figure 1). The series is turned into a regression table whose predictors are earlier values of the series. Response and predictors are standardised. The hyperparameters of a Gaussian process are set by maximising a log posterior with the Rprop optimiser. The fitted process then gives a predictive mean and standard deviation for any new row of predictors.

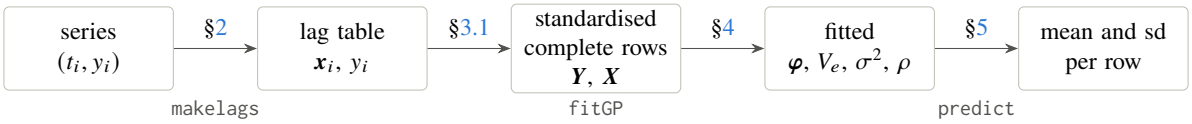



**Figure 1:** The four steps and the function that performs each.

In code, a forecast of the next four time steps is three calls:

```
lags = makelags(df, y="X", time="Time", E=E, tau=1, vtimestep=True)
fit  = fitGP(pd.concat([df, lags], axis=1), y="X", x=list(lags.columns), time="Time")
fore = makelags(df, y="X", time="Time", E=E, tau=1, vtimestep=True,
                forecast=True, Tdiff_fore=[1, 2, 3, 4])
pred = predict(fit, newdata=fore).outsampresults    # predmean, predfsd, predsd
```

That is how the RespiCast forecast used the port up to the round with origin date 14 October 2026. From the round of 21 October 2026 it calls `makelags` no more: `gpedm_direct.py` builds one table of fixed-step inputs per horizon and fits one Gaussian process to each (Section 6.1),

```
for h in (1, 2, 3, 4):
    train, fore, xcols = direct_design(z, h, range(E), first_week, season=True, diff=True)
    fit  = fitGP(train, y="y", x=xcols, time="Time")
    pred = predict(fit, newdata=fore).outsampresults
```

Sections 3 to 5 describe `fitGP` and `predict`, which both forecasts use; Section 6 describes the two forecasts.

**Agreement with R.** On the 36 series of the RespiCast round with origin date 14 October 2026 the port and R 4.4.1 return identical lag tables, iteration counts and fitted parameters. Forecast means agree to $4 \times 10^{-10}$ relative and forecast standard deviations to $1.1 \times 10^{-6}$ relative; that remainder is rounding in the matrix inverse of the worst-conditioned series.

**Conventions.** Rows are numbered from 1 in this document and from 0 in the code. Bold lower-case letters are vectors and bold capitals are matrices; $\circ$ is the elementwise product. Appendix A maps every

symbol to its name in the code.

## 2 Lags: from a series to a regression table

Let $y_1, \dots, y_n$ be the series, recorded at times $t_1 < \cdots < t_n$. Some values may be missing; $O \subseteq \{1, \dots, n\}$ is the set of rows with an observed value. The embedding dimension $E \geq 1$ and the lag step $\tau \geq 1$ are integers. `makelags` returns, for every row $i$, a vector of predictors $\boldsymbol{x}_i$ built from earlier rows; row $i$ of the regression table is then $(\boldsymbol{x}_i, y_i)$.

### 2.1 Fixed step

With `vtimestep=False` the lags are counted in rows:

$$\boldsymbol{x}_i = \big(y_{i-\tau},\ y_{i-2\tau},\ \dots,\ y_{i-E\tau}\big), \tag{2.1}$$

with an entry missing when its index is below 1. A missing $y_j$ makes every row that uses it incomplete, so one gap removes up to $E + 1$ rows from the fit. The series must have $n > E\tau$ rows.

With `forecast=True` the function returns the $\tau$ rows that follow the series,

$$\boldsymbol{x}^{\mathrm{f}}_r = \big(y_{n-\tau+r},\ y_{n-2\tau+r},\ \dots,\ y_{n-E\tau+r}\big), \qquad r = 1, \dots, \tau, \tag{2.2}$$

at times $t_n + r\,(t_n - t_{n-1})$.

### 2.2 Variable step

With `vtimestep=True` missing rows are skipped and the time that separates consecutive lags becomes a predictor. For row $i > E\tau$ define the rows the lags are taken from by

$$\ell_0(i) = i, \qquad \ell_k(i) = \max\big\{\, j \in O \ :\ j \leq \min\big(i - k\tau,\ \ell_{k-1}(i) - \tau\big)\big\}, \qquad k = 1, \dots, E. \tag{2.3}$$

Lag $k$ is the latest observed row that lies at least $k\tau$ rows before row $i$ and at least $\tau$ rows before lag $k - 1$. The predictors are the $E$ lagged values followed by the $E$ time gaps,

$$\boldsymbol{x}_i = \big(y_{\ell_1(i)}, \dots, y_{\ell_E(i)},\ \Delta_{i1}, \dots, \Delta_{iE}\big), \qquad \Delta_{ik} = t_{\ell_{k-1}(i)} - t_{\ell_k(i)}, \tag{2.4}$$

so the table has $d = 2E$ predictor columns, named `X_1`, ..., `X_E`, `Tdiff_1`, ..., `Tdiff_E` for a variable called `X`. For $\tau = 1$, definition (2.3) reduces to: lag $k$ is the $k$-th most recent observed row before row $i$.

Three details follow from (2.3).

- The first $E\tau$ rows get no lags at all, whatever the data.
- If the set in (2.3) is empty for some $k$, lag $k$ and all deeper lags are missing.
- The bounds count *rows*, while the gaps $\Delta_{ik}$ are differences of *times*. The two coincide only when the rows are equally spaced.

A row whose own value $y_i$ is missing still receives lags; it is dropped later, when the fit keeps complete rows only.

### 2.3 Forecast rows

With `forecast=True` and steps ahead $h_1, \ldots, h_H$ (`Tdiff_fore`), rows $n+1, \ldots, n+H$ are appended at times $t_{n+m} = t_n + h_m$, with unknown value, and (2.3) is applied to them with $O$ unchanged. Only these $H$ rows are returned. For $\tau = 1$ every forecast row has the same lagged values, the last $E$ observed values of the series, and the same gaps $\Delta_2, \ldots, \Delta_E$. The rows differ only in the first gap,

$$\Delta_{n+m,1} = t_n + h_m - t_{\ell_1}, \qquad \text{which is } h_m \text{ when the last row is observed.} \tag{2.5}$$

The forecast horizon therefore enters the model through one number, the column `Tdiff_1`. If `Tdiff_fore` is not given it defaults to the single step $\tau \min_i (t_{i+1} - t_i)$.

### 2.4 Several variables, several populations, long gaps

With $v$ variables, a row counts as observed when all $v$ are present; each variable contributes its $E$ lagged values at the same rows $\ell_k(i)$, and the $E$ gaps follow, so $d = E(v+1)$. With a population column the construction runs separately within each population. `Tdiff_max` sets every gap larger than it to missing, which removes those rows from the fit.

### 2.5 Example

Take $n = 10$ rows at times $t_i = i$, with $y_4$, $y_7$ and $y_8$ missing, $E = 2$ and $\tau = 1$. Figure 2 shows the lags of row 9 and of a forecast row; Table 1 lists the whole table as `makelags` returns it.

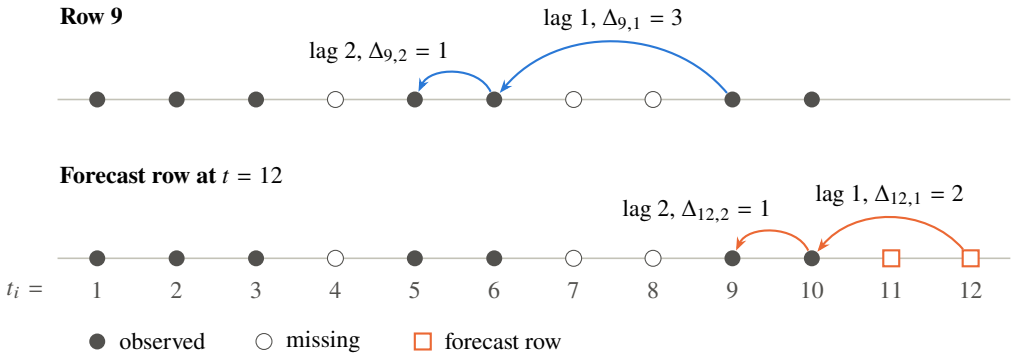



**Figure 2:** Variable-step lags for $E = 2$, $\tau = 1$; an arrow points from a row to the row its lag is taken from. Row 9 skips the two missing rows: its first lag is row 6, three time steps back, and its second lag is row 5. The forecast row at $t = 12$ takes the last two observed rows; only its first gap differs from that of the forecast row at $t = 11$.

## 3 The model

### 3.1 Standardising and selecting rows

`fitGP` receives the response column and $d$ predictor columns. With the default `scaling="global"` each column is centred and scaled by its own mean and standard deviation,

$$\tilde{y}_i = \frac{y_i - m_y}{s_y}, \qquad \tilde{x}_{ik} = \frac{x_{ik} - m_k}{s_k}, \tag{3.1}$$

where a column's mean and standard deviation (denominator: count minus one) are taken over all rows in which that column is present. `scaling="local"` does the same within each population, and `scaling="none"` uses the columns as given.

| $i$ | $y_i$ | y_1 | y_2 | Tdiff_1 | Tdiff_2 | use |
|---|---|---|---|---|---|---|
| 1 | $y_1$ | – | – | – | – | no lags ($i \le E\tau$) |
| 2 | $y_2$ | – | – | – | – | no lags ($i \le E\tau$) |
| 3 | $y_3$ | $y_2$ | $y_1$ | 1 | 1 | fitted |
| 4 | missing | $y_3$ | $y_2$ | 1 | 1 | dropped, no response |
| 5 | $y_5$ | $y_3$ | $y_2$ | 2 | 1 | fitted |
| 6 | $y_6$ | $y_5$ | $y_3$ | 1 | 2 | fitted |
| 7 | missing | $y_6$ | $y_5$ | 1 | 1 | dropped, no response |
| 8 | missing | $y_6$ | $y_5$ | 2 | 1 | dropped, no response |
| 9 | $y_9$ | $y_6$ | $y_5$ | 3 | 1 | fitted |
| 10 | $y_{10}$ | $y_9$ | $y_6$ | 1 | 3 | fitted |
| 11 | to predict | $y_{10}$ | $y_9$ | 1 | 1 | forecast row, $h = 1$ |
| 12 | to predict | $y_{10}$ | $y_9$ | 2 | 1 | forecast row, $h = 2$ |

**Table 1:** The lag table of the example. Five of the ten rows can be fitted. With fixed-step lags only row 3 would be complete.

The fit keeps the *complete rows*: those with a response and all $d$ predictors. Let there be $N$ of them, collected in $\boldsymbol{Y} \in \mathbb{R}^N$ and $\boldsymbol{X} \in \mathbb{R}^{N\times d}$; from here on $i, j = 1, \dots, N$ index these rows and $\boldsymbol{x}_i$ is row $i$ of $\boldsymbol{X}$. A constant column has $s_k = 0$, so (3.1) gives $0/0$ for every row and no row is complete. This is what happens to the gap columns of (2.4) when a series has no gap at all: every $\Delta_{ik}$ equals the sampling interval. `gpedm_py` raises an error in that case.

### 3.2 Prior over functions and likelihood

The standardised response is modelled as a smooth function of the predictors plus independent noise,

$$Y_i = f(\boldsymbol{x}_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, V_e), \qquad f \sim \mathcal{GP}(0, c), \tag{3.2}$$

with the squared-exponential covariance function, one coefficient per input,

$$c(\boldsymbol{x}, \boldsymbol{x}') = \sigma^2 \exp\Big(-\sum_{k=1}^{d} \varphi_k \,(x_k - x'_k)^2\Big)\, r, \qquad r = \begin{cases} 1 & \text{same population} \\ \rho & \text{different populations.} \end{cases} \tag{3.3}$$

The mean function is zero because the response has been centred. The coefficient $\varphi_k$ is an inverse squared length scale: $\exp(-\varphi_k u^2) = \exp\big(-u^2/2\ell_k^2\big)$ with $\ell_k = (2\varphi_k)^{-1/2}$, measured in standard deviations of input $k$. As $\varphi_k \to 0$ the function stops depending on input $k$, which is how the fit selects among the lags. Write $\boldsymbol{C}$ for the $N \times N$ matrix with entries $C_{ij} = c(\boldsymbol{x}_i, \boldsymbol{x}_j)$ and

$$\boldsymbol{\Sigma} = \boldsymbol{C} + V_e\, \boldsymbol{I}. \tag{3.4}$$

Then $\boldsymbol{Y} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Sigma})$, and the log likelihood of the hyperparameters $\boldsymbol{\theta} = (\varphi_1, \dots, \varphi_d, V_e, \sigma^2, \rho)$ is, up to a constant,

$$L(\boldsymbol{\theta}) = -\tfrac{1}{2}\, \boldsymbol{Y}^{\mathsf{T}} \boldsymbol{\Sigma}^{-1} \boldsymbol{Y} - \tfrac{1}{2} \log\det \boldsymbol{\Sigma}. \tag{3.5}$$

The code factorises $\boldsymbol{\Sigma} = \boldsymbol{R}^{\mathsf{T}}\boldsymbol{R}$ with $\boldsymbol{R}$ upper triangular, forms $\boldsymbol{\Sigma}^{-1}$ explicitly from the factor (LAPACK `dpotrf` and `dpotri`, as R does), and uses

$$\boldsymbol{a} = \boldsymbol{\Sigma}^{-1}\boldsymbol{Y}, \qquad \boldsymbol{Y}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{Y} = \boldsymbol{Y}^{\mathsf{T}}\boldsymbol{a}, \qquad \tfrac{1}{2}\log\det\boldsymbol{\Sigma} = \sum_{i=1}^{N} \log R_{ii}. \tag{3.6}$$

### 3.3 Priors

The hyperparameters have independent priors:

$$\pi(\varphi_k) \propto \exp\Big(-\frac{\varphi_k^2}{2\lambda}\Big), \quad \varphi_k > 0, \qquad \lambda = \big(2^{m-1}\big)^2 \frac{\pi}{2}, \tag{3.7}$$

$$V_e/V_{\max} \sim \text{Beta}(2,2), \qquad \sigma^2/\sigma^2_{\max} \sim \text{Beta}(2,2), \qquad \rho \sim \text{Beta}(2,2), \tag{3.8}$$

with $V_{\max} = \sigma^2_{\max} = 4.9999$. The half-normal prior (3.7) has mean $\sqrt{2\lambda/\pi} = 2^{m-1}$, where $m$ is the argument `modeprior` (default 1, so $\lambda = \pi/2$ and the prior mean of every $\varphi_k$ is 1). Up to a constant,

$$\log \pi(\boldsymbol{\theta}) = -\frac{1}{2\lambda}\sum_{k=1}^{d} \varphi_k^2 + \log\frac{V_e}{V_{\max}} + \log\Big(1-\frac{V_e}{V_{\max}}\Big) + \log\frac{\sigma^2}{\sigma^2_{\max}} + \log\Big(1-\frac{\sigma^2}{\sigma^2_{\max}}\Big) + \log\rho + \log(1-\rho). \tag{3.9}$$

### 3.4 Unconstrained parameters

The optimiser works on a vector $\boldsymbol{p} \in \mathbb{R}^{d+3}$ without constraints (Table 2):

$$\varphi_k = e^{p_k}, \qquad \theta = \theta_{\min} + \frac{\theta_{\max} - \theta_{\min}}{1 + e^{-p}} \quad \text{for } \theta \in \{V_e, \sigma^2, \rho\}, \tag{3.10}$$

with derivatives

$$\frac{d\varphi_k}{dp_k} = \varphi_k, \qquad \frac{d\theta}{dp} = (\theta - \theta_{\min})\Big(1 - \frac{\theta - \theta_{\min}}{\theta_{\max} - \theta_{\min}}\Big). \tag{3.11}$$

The prior (3.9) is a density in $\boldsymbol{\theta}$ and no Jacobian term is added for the change of variables, so the fit targets the posterior mode in $\boldsymbol{\theta}$, not in $\boldsymbol{p}$.

| | meaning | range | prior | start | in $\boldsymbol{p}$ |
|---|---|---|---|---|---|
| $\varphi_k$ | inverse squared length scale of input $k$ | $(0, \infty)$ | half-normal, mean $2^{m-1}$ | 0.1 | $\log \varphi_k$ |
| $V_e$ | noise variance | $(10^{-4}, 4.9999)$ | Beta(2, 2) on $V_e/4.9999$ | 0.001 | logistic |
| $\sigma^2$ | variance of $f$ | $(10^{-4}, 4.9999)$ | Beta(2, 2) on $\sigma^2/4.9999$ | 0.999 | logistic |
| $\rho$ | correlation between populations | $(10^{-4}, 1)$ | Beta(2, 2) | 0.5 | logistic |

**Table 2:** The $d + 3$ hyperparameters. Variances are on the standardised scale, where the response has variance 1. With a single population $\rho$ is not estimated: it is held at 0.5 and has no effect.

## 4 Fitting

### 4.1 Objective and gradient

The fit minimises the negative log posterior

$$F(\boldsymbol{p}) = -\Big[L\big(\boldsymbol{\theta}(\boldsymbol{p})\big) + \log \pi\big(\boldsymbol{\theta}(\boldsymbol{p})\big)\Big], \tag{4.1}$$

returned as `nllpost`; the fit statistic `ln_post` is $-F$ at the final point. For any hyperparameter $\theta$ the derivative of (3.5) is [4, eq. 5.9]

$$\frac{\partial L}{\partial \theta} = \operatorname{tr}\Big(\boldsymbol{Q}\,\frac{\partial \boldsymbol{\Sigma}}{\partial \theta}\Big) = \sum_{i,j} Q_{ij}\,\frac{\partial \Sigma_{ij}}{\partial \theta}, \qquad \boldsymbol{Q} = \tfrac{1}{2}\big(\boldsymbol{a}\boldsymbol{a}^{\mathsf{T}} - \boldsymbol{\Sigma}^{-1}\big), \tag{4.2}$$

and from (3.3), with $D_{ij}^{(k)} = (X_{ik} - X_{jk})^2$ and $S_{ij} = 1$ when rows $i$ and $j$ belong to the same population and 0 otherwise,

$$\frac{\partial \boldsymbol{\Sigma}}{\partial \varphi_k} = -\boldsymbol{D}^{(k)} \circ \boldsymbol{C}, \qquad \frac{\partial \boldsymbol{\Sigma}}{\partial V_e} = \boldsymbol{I}, \qquad \frac{\partial \boldsymbol{\Sigma}}{\partial \sigma^2} = \frac{\boldsymbol{C}}{\sigma^2}, \qquad \frac{\partial \boldsymbol{\Sigma}}{\partial \rho} = \frac{\boldsymbol{C} \circ (\boldsymbol{1}\boldsymbol{1}^{\mathsf{T}} - \boldsymbol{S})}{\rho}. \tag{4.3}$$

The prior (3.9) contributes

$$\frac{\partial \log \pi}{\partial \varphi_k} = -\frac{\varphi_k}{\lambda}, \qquad \frac{\partial \log \pi}{\partial V_e} = \frac{1}{V_e} - \frac{1}{V_{\max} - V_e}, \qquad \frac{\partial \log \pi}{\partial \rho} = \frac{1}{\rho} - \frac{1}{1-\rho}, \tag{4.4}$$

and the same form as for $V_e$ holds for $\sigma^2$. The gradient that `getlikegrad` returns, and that drives the optimiser, is

$$g_k = -\left(\gamma_k \frac{\partial L}{\partial \theta_k} + \frac{\partial \log \pi}{\partial \theta_k}\right) \frac{d\theta_k}{dp_k}, \qquad \gamma_k = \begin{cases} \frac{1}{2} & \text{for } \varphi_1, \ldots, \varphi_d \\ 1 & \text{for } V_e,\ \sigma^2,\ \rho. \end{cases} \tag{4.5}$$

**The factor $\gamma_k = \frac{1}{2}$.** The exact gradient of (4.1) has $\gamma_k = 1$ for every parameter. GPEDM multiplies the likelihood part of the $\varphi$ entries by a further one half, and the port does the same so that its results match R. Finite differences of $F$ confirm it: for every $\varphi_k$ whose gradient is clear of rounding noise the ratio of the coded to the numerical likelihood gradient is 0.50, and it is 1.00 for $V_e$ and $\sigma^2$ (`gpedm_py_validation/check_gradient.py`). Section 4.3 states what this does to the fit.

## 4.2 The optimiser

`fmingrad_Rprop` is Rprop [5] in the variant without backtracking, often written Rprop$^-$ [6]. Each parameter has its own step size. A step moves every parameter against the sign of its gradient entry by its step size; the size of the gradient is never used. A step size grows by a factor 1.2 when its gradient entry kept its sign and halves when the sign flipped (Algorithm 1).

**Algorithm 1:** `fmingrad_Rprop`

**Input:** start $\boldsymbol{p}$ from Table 2; data $\boldsymbol{Y}$, $\boldsymbol{X}$

$\boldsymbol{\delta} \leftarrow 0.1 \cdot \boldsymbol{1}$; $t \leftarrow 0$; $q \leftarrow 10$; $(F, \boldsymbol{g}) \leftarrow$ objective (4.1) and gradient (4.5) at $\boldsymbol{p}$

**while** $\|\boldsymbol{g}\|_2 > 10^{-4}$ ***and*** $t < 200$ ***and*** $q > 10^{-7}$ **do**

  $\boldsymbol{p}' \leftarrow \boldsymbol{p} - \operatorname{sign}(\boldsymbol{g}) \circ \boldsymbol{\delta}$

  $(F', \boldsymbol{g}') \leftarrow$ objective and gradient at $\boldsymbol{p}'$

  $q \leftarrow |F'/F - 1|$

  **for** $k = 1, \ldots, d + 3$ **do**

    $\delta_k \leftarrow \operatorname{clip}\big(\eta_k\, \delta_k,\ 10^{-6},\ 50\big)$, $\eta_k = 1.2$ if $g_k g'_k > 0$, $0.5$ if $g_k g'_k < 0$, $1$ if $g_k g'_k = 0$

  $(\boldsymbol{p}, F, \boldsymbol{g}) \leftarrow (\boldsymbol{p}', F', \boldsymbol{g}')$; $t \leftarrow t + 1$

**Output:** $\boldsymbol{p}$, and with it $\boldsymbol{\theta}$, $\boldsymbol{\Sigma}$, $\boldsymbol{\Sigma}^{-1}$ and $\boldsymbol{a}$ at the final point

## 4.3 What the optimiser maximises

(i) **The effective prior on $\boldsymbol{\varphi}$.** Because only signs are used, multiplying gradient entries by positive constants does not change the path. By (4.5), $\boldsymbol{g} = \boldsymbol{\Gamma}\, \nabla \tilde{F}$ with $\boldsymbol{\Gamma} = \operatorname{diag}(\frac{1}{2}, \ldots, \frac{1}{2}, 1, 1, 1)$ and

$$\tilde{F}(\boldsymbol{p}) = -\Big[L + 2 \log \pi(\boldsymbol{\varphi}) + \log \pi(V_e) + \log \pi(\sigma^2) + \log \pi(\rho)\Big]. \tag{4.6}$$

The iterates are therefore those of Rprop applied to $\tilde{F}$: the fit is the posterior mode with the half-normal prior on each $\varphi_k$ counted twice, that is with variance parameter $\lambda/2$ in place of $\lambda$. The identity was checked numerically at the fitted point of the two series of Section 6. The reported `nllpost` is still $F$, and the stopping rule is evaluated on $F$ and on $\boldsymbol{g}$.

(ii) **Stopping.** The loop ends when the gradient norm falls below $10^{-4}$, when the relative change $q$ of the objective falls below $10^{-7}$, or after 200 iterations. $q$ is a ratio of successive values of $F$, which has no fixed sign, so $q$ is large whenever $F$ passes near zero (Figure 4, iterations 14 and 15).

(iii) **No step is rejected.** There is no line search: $F$ may rise from one iteration to the next, and the final point is the last iterate, not the best one. In the fit of Section 6 $F$ rises at iterations 12, 27, 28 and 38, the last one.

(iv) **Determinism.** The start is fixed and nothing is random, so a fit is a function of the data. The path depends on the data only through gradient signs, which is why the port reproduces the parameters of R exactly although its arithmetic differs in the last digit.

(v) **Inputs switched off.** A step size can reach 50 on the log scale, so $p_k$ can run far below zero in a few iterations. Below about $-745$, $\varphi_k = e^{p_k}$ is exactly 0 in double precision and input $k$ is removed from the covariance.

(vi) **Fixed parameters.** A parameter fixed through `fixedpars` or `rhofixed` has its likelihood derivative set to zero, but its prior derivative stays in $\boldsymbol{g}$ and so in the gradient norm of the stopping rule. The value used in $\boldsymbol{\Sigma}$ is always the fixed one.

# 5 Prediction

All predictions are computed on the standardised scale from the quantities stored at the end of the fit, $\boldsymbol{C}$, $\boldsymbol{\Sigma}$, $\boldsymbol{\Sigma}^{-1}$ and $\boldsymbol{a} = \boldsymbol{\Sigma}^{-1}\boldsymbol{Y}$, and then transformed back (Section 5.6). Three numbers are returned per row: the mean, the standard deviation of the function $f$ alone (`predfsd`) and the standard deviation of a new observation (`predsd`), which adds the noise variance $V_e$.

## 5.1 Fitted values

For the rows used in the fit, the posterior mean and covariance of $f$ are

$$\hat{\boldsymbol{f}} = \boldsymbol{C}\boldsymbol{a}, \qquad \operatorname{Cov}(\boldsymbol{f} \mid \boldsymbol{Y}) = \boldsymbol{C} - \boldsymbol{C}\boldsymbol{\Sigma}^{-1}\boldsymbol{C}, \tag{5.1}$$

and `insampresults` holds $\hat{f}_i$, $\sqrt{v_i}$ and $\sqrt{v_i + V_e}$ with $v_i$ the $i$-th diagonal entry of the covariance.

## 5.2 New inputs

A new row of predictors is standardised with the means and standard deviations of the training data (3.1). With $\boldsymbol{c}_* = \big(c(\boldsymbol{x}_*, \boldsymbol{x}_1), \ldots, c(\boldsymbol{x}_*, \boldsymbol{x}_N)\big)^{\mathsf{T}}$,

$$\mu_* = \boldsymbol{c}_*^{\mathsf{T}}\boldsymbol{a}, \qquad v_* = \sigma^2 - \boldsymbol{c}_*^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{c}_*, \tag{5.2}$$

and the row gets mean $\mu_*$, `predfsd` $\sqrt{v_*}$ and `predsd` $\sqrt{v_* + V_e}$. A new row with a missing predictor gets missing predictions. Far from all training rows $\boldsymbol{c}_* \to \boldsymbol{0}$, so $\mu_* \to 0$ and $v_* \to \sigma^2$: the prediction returns to the prior, which on the original scale is the mean of the series with standard deviation $s_y\sqrt{\sigma^2 + V_e}$.

## 5.3 Leave-one-out

`predictmethod="loo"` predicts each fitted row $i$ from the others. With exclusion radius $r$ (`exclradius`), the rows left out are those of the same population within $r$ positions of $i$ in the fitted data; let $K$ be the remaining rows. Then

$$\mu_{-i} = \boldsymbol{C}_{iK}\,\boldsymbol{\Sigma}_{KK}^{-1}\,\boldsymbol{Y}_K, \qquad v_{-i} = \sigma^2 - \boldsymbol{C}_{iK}\,\boldsymbol{\Sigma}_{KK}^{-1}\,\boldsymbol{C}_{Ki}, \tag{5.3}$$

computed with one matrix inverse per row. For $r = 0$ these have the closed form [4, Sec. 5.4.2]

$$\mu_{-i} = Y_i - \frac{a_i}{[\boldsymbol{\Sigma}^{-1}]_{ii}}, \qquad v_{-i} + V_e = \frac{1}{[\boldsymbol{\Sigma}^{-1}]_{ii}}, \tag{5.4}$$

which the loop reproduces to better than $10^{-12}$ on the series of Section 6. The fit statistic `lnL_LOO` is the leave-one-out log predictive density from (5.4), without its constant:

$$\texttt{lnL_LOO} = \sum_{i=1}^{N} \Big( \tfrac{1}{2} \log [\boldsymbol{\Sigma}^{-1}]_{ii} - \tfrac{1}{2} \frac{a_i^2}{[\boldsymbol{\Sigma}^{-1}]_{ii}} \Big). \tag{5.5}$$

The hyperparameters are not refitted when rows are left out.

### 5.4 Leave-time-out

`predictmethod="lto"` leaves out whole time points, in all populations at once. Let $\tau_1, \ldots, \tau_T$ be the distinct times of the fitted rows in order of first appearance. For time point $u$ the training set $K$ is every row whose time is not among $\tau_{u-r}, \ldots, \tau_{u+r}$, and all rows at $\tau_u$ are predicted from $K$ with the formulas above. The radius counts positions in the list of time points, not units of time.

### 5.5 Sequential

`predictmethod="sequential"` predicts each time point from the earlier ones, with the fitted hyperparameters. Starting from $\boldsymbol{m}^{(1)} = \boldsymbol{0}$ and $\boldsymbol{S}^{(1)} = \boldsymbol{C}$, and with $I_u$ the rows at time point $\tau_u$, the update for $u = 1, \ldots, T-1$ is

$$\boldsymbol{K}_u = \big( \boldsymbol{S}^{(u)}_{I_u I_u} + V_e \boldsymbol{I} \big)^{-1}, \tag{5.6}$$

$$\boldsymbol{m}^{(u+1)} = \boldsymbol{m}^{(u)} + \boldsymbol{S}^{(u)}_{\cdot I_u} \boldsymbol{K}_u \big( \boldsymbol{Y}_{I_u} - \boldsymbol{m}^{(u)}_{I_u} \big), \qquad \boldsymbol{S}^{(u+1)} = \boldsymbol{S}^{(u)} - \boldsymbol{S}^{(u)}_{\cdot I_u} \boldsymbol{K}_u \boldsymbol{S}^{(u)}_{I_u \cdot}. \tag{5.7}$$

A row $j \in I_u$ is predicted from the data of time points $\tau_1, \ldots, \tau_{u-1}$: its mean is $m^{(u)}_j$ and, for $u \geq 2$, its `predsd` is $\sqrt{|S^{(u)}_{jj}| + V_e}$ (Section 7 has the case $u = 1$). `predfsd` is $\sqrt{\texttt{predsd}^2 - V_e}$.

### 5.6 Back-transformation and fit statistics

With global scaling a standardised mean $\mu$ and variance $v$ become

$$\texttt{predmean} = m_y + s_y \mu, \qquad \texttt{predfsd} = s_y \sqrt{v}, \qquad \texttt{predsd} = s_y \sqrt{v + V_e}, \tag{5.8}$$

and with local scaling the population's own $m_y$ and $s_y$ are used. `insampfitstats` reports, besides `ln_post`, `ln_prior` and `lnL_LOO`: $R^2 = 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$ and the root mean squared error of the fitted values on the original scale; the effective number of parameters $\texttt{df} = \operatorname{tr}(\boldsymbol{C}\boldsymbol{\Sigma}^{-1})$; $\texttt{SS} = \boldsymbol{Y}^{\mathsf{T}} \boldsymbol{\Sigma}^{-1} \boldsymbol{Y}$; and $\texttt{logdet} = \frac{1}{2} \log\det \boldsymbol{\Sigma}$.

## 6 Use in the RespiCast forecast

`respi_edm.run_analysis` forecasts each country and target separately. Two models have been submitted. Up to the round with origin date 14 October 2026 it was one Gaussian process on variable-step lags whose four horizons differ only in a time-gap input (Section 6.4). From the round of 21 October 2026 it is one Gaussian process per horizon on the log scale, with the time of year as input (Sections 6.1 and 6.2), chosen on a replay of 85 past rounds (Section 6.3).

## 6.1 The forecast from 21 October 2026

The code is `gpedm_direct.forecast`, called by `respi_edm.run_analysis` with its default `method="direct"`.

**Which series.** The weekly values of one country are laid out by date, $x_1, \ldots, x_n$, from its first to its last observation; a week without a report is missing, and no week is blanked. Let $k$ be the number of weeks from the last observation $x_n$ to the newest week of the round, the week before the target week of horizon 1. The series is forecast if

- $k \le 1$ (`respi_edm.MAX_WEEKS_MISSING`), judged by the latest date in the data, not by the first row: the hub lists the rows of the UK nations and of Swiss ILI oldest first, and the test on the first row skipped those series up to 14 October 2026 although their newest week was present;
- at least 30 weeks are observed, and every horizon has at least 30 complete training rows.

Skipped series are named in a printed message.

**Scale.** The model works with

$$z_i = \log(x_i + c), \qquad c = \max\bigl(1,\ \bar{x}/10\bigr), \tag{6.1}$$

where $\bar{x}$ is the mean of the observed values and $c$ is rounded to six significant digits. An offset in proportion to the level of the series keeps a zero, or a very small value, in a series of large values from becoming an extreme point on the log scale. For Belgian ARI, $c = 94.09$.

**Embedding dimension.** $E \le 8$ is chosen by the false-nearest-neighbour test, as before (the random generator is seeded with 42, because the mutual-information step jitters its input).

**One regression per horizon.** The forecast for the hub's horizon $h = 1, \ldots, 4$ lies $H = k + h$ weeks after the last observation. Its training rows are the weeks $s$ with

$$\boldsymbol{x}_s^{(H)} = \bigl(z_s,\ \tilde{z}_{s-1},\ \ldots,\ \tilde{z}_{s-E+1},\ \sin\theta_{s+H},\ \cos\theta_{s+H}\bigr), \qquad y_s^{(H)} = z_{s+H} - z_s, \tag{6.2}$$

where $\theta_t = 2\pi\,(\text{day of the year of the last day of week } t-1)/365.25$ is the time of year of the target week, and $\tilde{z}$ is $z$ with missing weeks filled by linear interpolation between observed weeks, which for a lag lie at or before week $s$. A row is complete when $z_s$ and $z_{s+H}$ are observed and $s \ge E$. The $d = E + 2$ inputs are all on a fixed step, so no column is constant and no week has to be blanked (compare Section 3.1). `fitGP` is called with one population, global scaling and the default priors and start, once for each $H$, so each horizon has its own hyperparameters.

**Forecast.** The forecast row is the last week, $\boldsymbol{x}_n^{(H)}$. `predict` (5.2) gives the mean $\mu_H$ and the standard deviation $\sigma_H$ (`predsd`, which includes $V_e$) of the change $z_{n+H} - z_n$. The forecast distribution is

$$z_{n+H} \sim \mathcal{N}\bigl(z_n + \mu_H,\ \sigma_H^2\bigr), \qquad \hat{x}_q = \max\Bigl(0,\ \exp\bigl(z_n + \mu_H + \Phi^{-1}(q)\,\sigma_H\bigr) - c\Bigr), \tag{6.3}$$

a log-normal shifted by $c$ and cut at zero, with $\Phi$ the standard normal distribution function. The submission holds, for each horizon, the 23 hub quantiles $\hat{x}_q$ and the median $\hat{x}_{0.5}$.

> **As coded.** `direct_design` returns every week $s = 1, \ldots, n - H$ as a row; `fitGP` drops the incomplete ones (Section 3.1). A lag longer than the series stays missing, so such an $E$ gives no forecast rather than an error. A series with $k = 1$ is fitted with $H = 2, \ldots, 5$ and its four forecasts are submitted as the hub's horizons 1 to 4. The table that `run_analysis` returns holds, per horizon, $k$ (`weeks_missing`), $E$, the transform `log:`$c$, $z_n + \mu_H$,

$\sigma_H$ and the median.

## 6.2 One forecast in numbers

Belgium ARI on the round with origin date 14 October 2026 (the same data as in Section 6.4.1): $n = 224$ weeks from 26 June 2022, none missing, $\bar{x} = 940.9$, $c = 94.09$, $E = 3$, last value $x_n = 1071.3$. Table 3 lists the four fits.

| horizon $H$ | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| complete rows $N$ | 221 | 220 | 219 | 218 |
| optimiser iterations | 58 | 68 | 69 | 74 |
| length scale $\ell$ of $z_s$ | 1.75 | 1.44 | 1.75 | 1.81 |
| $z_{s-1}$ | 2.15 | 4.33 | 2.69 | 2.53 |
| $z_{s-2}$ | – | 9.18 | 3.07 | 2.97 |
| $\sin\theta$ | 0.58 | 0.50 | 0.49 | 0.49 |
| $\cos\theta$ | 1.04 | 0.78 | 0.79 | 0.86 |
| noise variance $V_e$ | 0.541 | 0.307 | 0.194 | 0.149 |
| variance of $f$, $\sigma^2$ | 2.83 | 2.41 | 2.32 | 1.87 |
| s.d. of the change $z_{s+H} - z_s$, $s_y$ | 0.184 | 0.244 | 0.313 | 0.370 |
| forecast change $\mu_H$ | +0.043 | +0.024 | −0.031 | −0.076 |
| forecast s.d. $\sigma_H$ | 0.148 | 0.151 | 0.167 | 0.175 |
| median $\hat{x}_{0.5}$ | 1122.6 | 1099.2 | 1035.7 | 985.8 |
| 95% interval | 816–1532 | 794–1509 | 720–1473 | 672–1429 |

**Table 3:** The four fits of the forecast for Belgium ARI. Length scales are in standard deviations of the input (–: $\varphi = 0$, the input is switched off). The time of year has the shortest length scales: the fits learn above all how the weekly change depends on the season. The forecast standard deviation is a small fraction of $s_y$, the spread of the change over the whole series.

In Slovakia ($c = 145.55$, $E = 3$, last value 1282.5) the medians are 1359.7, 1553.4, 1619.4 and 1495.3, where the previous model gave 1257.8 at every horizon (Figure 3).

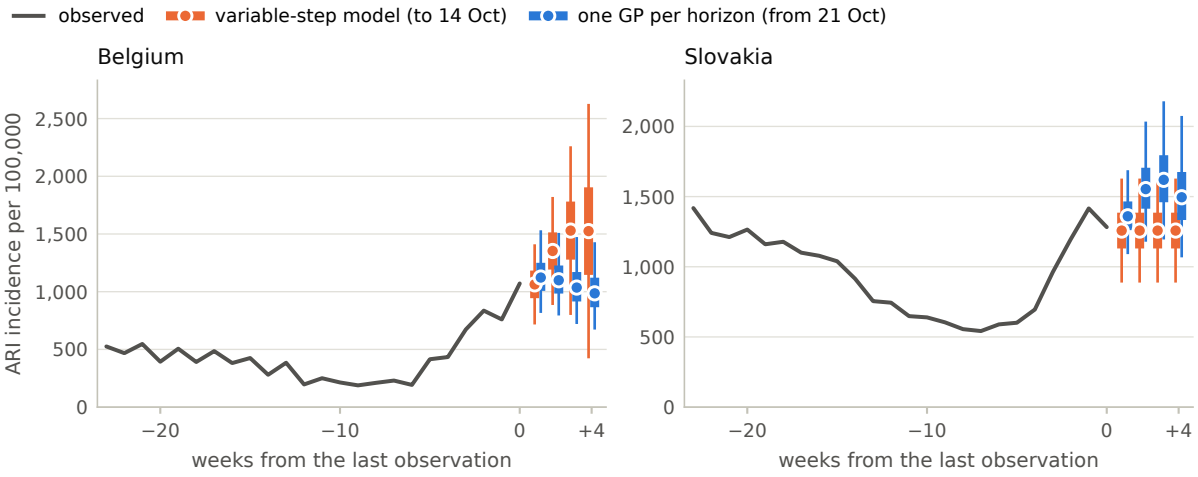



**Figure 3:** The two ARI series of Figure 5, forecast on the same data by the model submitted up to 14 October 2026 (orange) and by the one submitted from 21 October 2026 (blue): median, 50% and 95% intervals.

### 6.3 Why this model: a replay of 85 rounds

Each model variant was run on every past round from origin date 16 October 2024 to 7 October 2026, on the hub data as published on the Friday before the origin date, and scored like the hub scores the submitted models: the weighted interval score (WIS) against the values published later, turned into a rank score $(\mathrm{WIS}_{\mathrm{worst}} - \mathrm{WIS})/(\mathrm{WIS}_{\mathrm{worst}} - \mathrm{WIS}_{\mathrm{best}})$ among the other models of the hub for the same country, target and week (about 11 models). A score of 1 is the best model on that forecast. The mean is over 8,844 forecasts of 43 series; the intervals are 95% block-bootstrap intervals over origin dates of the difference from the submitted model. The code is in `gpedm_method_tests/`.

| variant | rank score | difference (95% interval) |
|---|---|---|
| hub quantile baseline | 0.697 | |
| model submitted up to 14 October (variable step, incidence scale) | 0.690 | 0 |
| + time of year as input | 0.724 | +0.034 (0.021 to 0.049) |
| + log scale, $c = 1$ | 0.739 | +0.049 (0.031 to 0.067) |
| + both | 0.776 | +0.086 (0.069 to 0.107) |
| one Gaussian process per horizon | 0.699 | +0.009 (−0.009 to 0.029) |
| + log scale, $c = 1$ | 0.772 | +0.082 (0.067 to 0.098) |
| + log scale + time of year | 0.795 | +0.105 (0.080 to 0.130) |
| + the same, change as response | 0.802 | +0.112 (0.087 to 0.137) |
| + offset $c = \max(1, \bar{x}/10)$: **submitted from 21 October** | **0.807** | +0.117 (0.092 to 0.144) |
| hub ensemble | 0.807 | |

**Table 4:** Mean rank score of each variant over the same 8,844 forecasts. The offset $c = \bar{x}/10$ adds 0.005 (0.003 to 0.008) to $c = 1$; it mostly repairs Luxembourg ILI, where single zeros in a series of large values become outliers with $c = 1$.

Tried and not adopted, with the difference from the submitted model:

- **Two embedding dimensions mixed**, one short (1 to 8 weeks) and one long (26 to 78 weeks), as the average of the two forecasts' quantiles: every pair with a short $E$ of 2 to 6 scores 0.793 to 0.799, the best pair (3 and 53) −0.009 (−0.021 to 0.004). A year-long $E$ alone scores 0.73.
- **Neighbouring countries as inputs** (six forms: their mean level, each neighbour, all other countries, their growth, their level relative to the country's own): −0.005 to −0.001.
- **The exact gradient of** $\varphi$ instead of GPEDM's halved one (Section 7): no change.
- **One Gaussian process for all countries**, with the country as population: +0.013 (0.006 to 0.020) over the same model fitted to each country alone, on every second round and with $E = 3$ and the last 208 weeks; not yet tried with the offset $c = \bar{x}/10$.

**Series behind by one week.** Forecasting a series from a last value that is $k$ weeks old costs accuracy. Table 5 compares the series the run forecast up to 14 October with those it skipped. The model stays level with the hub ensemble on the skipped series, but they are harder, so submitting them lowers the model's mean rank score from 0.807 to 0.800 (−0.0075, −0.011 to −0.003) while adding a fifth more forecasts. The run allows $k \leq 1$ from 21 October 2026.

### 6.4 The model submitted up to 14 October 2026

`respi_edm.run_analysis(method="vt")` still computes it (with `backend="r"` or `"python"`).

1. The weekly incidence values are laid out as a vector $y_1, \ldots, y_n$ with time $t_i = i - 1$; weeks without a report are missing.

| series on the origin date | series-dates | ranked forecasts | rank score | hub ensemble |
|---|---|---|---|---|
| newest week in the first row (submitted to 14 October) | 2,399 | 8,844 | 0.807 | 0.797 |
| newest week present, rows listed oldest first | 169 | 283 | 0.752 | 0.724 |
| last value one week old ($k = 1$) | 444 | 1,445 | 0.763 | 0.762 |
| last value two weeks old ($k = 2$, not submitted) | 149 | 418 | 0.747 | 0.709 |

**Table 5:** Rank score of the submitted model by how current the series was on the origin date, with the hub ensemble's rank score on the same forecasts. The first three rows are what the run submits from 21 October 2026.

2. The 21st value is set to missing (index 20 in the code). Without at least one gap every `Tdiff` column is constant and no row survives (Section 3.1).
3. The embedding dimension $E \le 8$ is chosen by a false-nearest-neighbour test. That test is run at a lag picked by mutual information, while the lags below are built with $\tau = 1$.
4. Variable-step lags with $\tau = 1$ give $d = 2E$ predictors (2.4). One population, global scaling, default priors and start.
5. Forecast rows for $h = 1, 2, 3, 4$ weeks ahead (2.5) are predicted with (5.2), giving $\mu_h =$ `predmean` and $\sigma_h =$ `predsd`.
6. The submission holds, for each horizon, the median $\max(0, \mu_h)$ and the 23 hub quantiles $\max\big(0,\ \mu_h + \Phi^{-1}(q)\,\sigma_h\big)$, with $\Phi$ the standard normal distribution function.

Only series whose first row is the newest week are forecast. The data step of this path lays the weeks out on a year × week-number grid. It fails on a series that contains an ISO week 53 (the first is the week ending 3 January 2027), and in a series that never reports some week numbers it moves the summer gap to the turn of the year. The model of Section 6.1 lays the weeks out by date and has neither problem.

### 6.4.1 One fit in numbers

The numbers of this section, the next and Section 6.2 are produced by `gpedm_py_docs/make_numbers.py`.

Belgium, acute respiratory infections (ARI), origin date 14 October 2026: $n = 224$ weeks, one gap (the blanked week), $E = 3$, so $d = 6$ and $N = 220$ complete rows. The optimiser stops after 38 iterations on the relative-change rule, with $F = -73.27$ and a gradient norm still at $1.1 \times 10^{-2}$ (Figure 4). Table 6 lists the result.

| input | $\varphi_k$ | $\ell_k$ | quantity | value |
|---|---|---|---|---|
| `X_1` (last value) | $2.5 \times 10^{-2}$ | 4.5 | noise variance $V_e$ | 0.159 |
| `X_2` | $7.1 \times 10^{-4}$ | 26.5 | variance of $f$, $\sigma^2$ | 3.66 |
| `X_3` | $2.1 \times 10^{-3}$ | 15.4 | $m_y$, $s_y$ (per 100,000) | 940.4, 440.7 |
| `Tdiff_1` | $4.1 \times 10^{-4}$ | 35.0 | in-sample $R^2$ | 0.848 |
| `Tdiff_2` | $1.5 \times 10^{-3}$ | 18.5 | effective parameters `df` | 6.35 |
| `Tdiff_3` | $6.5 \times 10^{-195}$ | – | `lnL_LOO` | 88.1 |

**Table 6:** Fitted hyperparameters for Belgium ARI. Length scales $\ell_k = (2\varphi_k)^{-1/2}$ are in standard deviations of the input. All are several standard deviations long, so the fitted function varies slowly over the range of the data; the third gap is switched off.

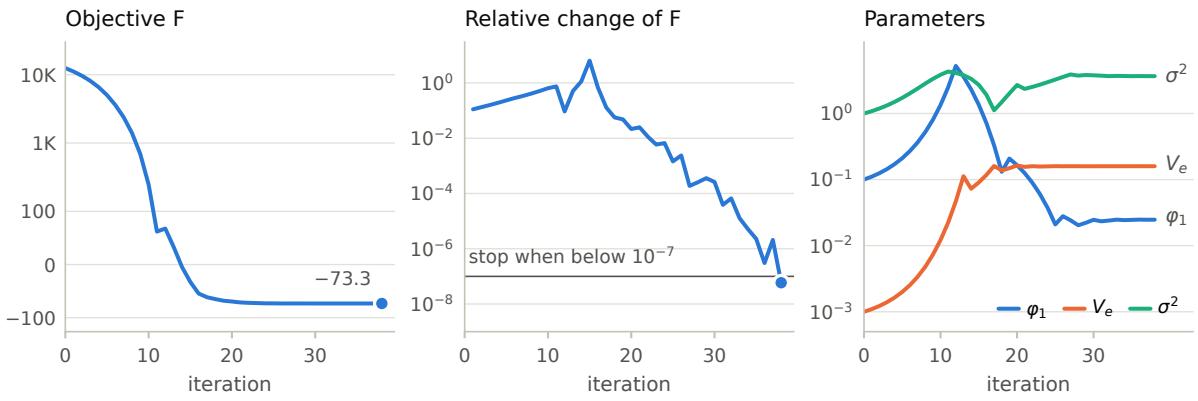



**Figure 4:** The optimiser on Belgium ARI. Left: the objective, on a symmetric log scale, falls from 12,497 to −73.3. Centre: its relative change, the quantity that ends the loop; the peak at iterations 14 and 15 is $F$ passing through zero. Right: three of the eight free parameters on their natural scale.

### 6.4.2 What the time gap does to the four horizons

The four forecast rows are identical except for the first gap, $\Delta_1 = h$ (2.5). Let $\varphi_\Delta$ be the coefficient of the column `Tdiff_1`, $s_\Delta$ its standard deviation, and $\Delta_j$ its value in fitted row $j$. Because the covariance (3.3) is a product over inputs, the covariance of the horizon-$h$ row with fitted row $j$ is the horizon-1 covariance times a factor that depends only on the gap of row $j$:

$$c_{*j}(h) = c_{*j}(1)\,\kappa_h(\Delta_j), \qquad \kappa_h(\Delta) = \exp\!\Big(-\varphi_\Delta\,\frac{(h-\Delta)^2-(1-\Delta)^2}{s_\Delta^2}\Big). \tag{6.4}$$

The standardised forecast mean (5.2) is then a sum over the gap values present in the fitted rows,

$$\mu(h) = \sum_{\Delta} \kappa_h(\Delta)\,T_\Delta, \qquad T_\Delta = \sum_{j:\,\Delta_j=\Delta} c_{*j}(1)\,a_j, \tag{6.5}$$

where $T_\Delta$ is what the rows with gap $\Delta$ contribute to the one-week forecast. Three regimes follow.

- $\varphi_\Delta = 0$**.** Then $\kappa_h \equiv 1$ and the forecast, mean and standard deviation, is the same at every horizon. In 20 of the 36 series of the round the fitted $\varphi_\Delta$ is below $10^{-89}$ and this is what the submission contains.
- $\varphi_\Delta > 0$**.** Rows with gap 1 are down-weighted by $\kappa_h(1) = \exp\!\big(-\varphi_\Delta(h-1)^2/s_\Delta^2\big)$. The scale $s_\Delta$ is small because nearly every fitted gap equals 1. When the blanked week is the only gap, the column holds $M-1$ ones and a single two over the $M$ rows where it is defined, its variance is exactly $1/M$, and
$$\kappa_h(1) = \exp\!\big(-\varphi_\Delta\,M\,(h-1)^2\big)\,: \tag{6.6}$$
a forecast row at horizon $h$ lies $(h-1)\sqrt{M}$ standard deviations from all but one of the fitted rows. For $M = 221$ that is 15, 30 and 45 standard deviations for $h = 2, 3, 4$.
- $\varphi_\Delta M \gg 1$**.** Then $\kappa_h(1) \approx 0$ for $h \geq 2$. The two-week forecast rests on the rows with gap 2 alone. For $h \geq 3$ no fitted row is close, $\mu(h) \to 0$ and $v(h) \to \sigma^2$: the forecast is the mean of the series with the prior standard deviation $s_y\sqrt{\sigma^2 + V_e}$.

Table 7 evaluates (6.5) for Belgium, where $\varphi_\Delta M = 4.08 \times 10^{-4} \times 221 = 0.090$; the sum reproduces the forecasts of the code to all printed digits. The single row that follows the blanked week contributes +3.77 to a one-week mean of 0.28, against −3.49 from the other 219 rows together, and its weight relative to theirs grows with the horizon. The forecasts for weeks 2 to 4 hinge on that one row, and so on which week was blanked. In Slovakia $\varphi_\Delta = 9 \times 10^{-282}$ and all four forecasts coincide (Figure 5).

| | rows | $T_\Delta$ | $\kappa_1$ | $\kappa_2$ | $\kappa_3$ | $\kappa_4$ |
|---|---|---|---|---|---|---|
| fitted rows with gap $\Delta = 1$ | 219 | −3.494 | 1 | 0.914 | 0.697 | 0.444 |
| fitted row with gap $\Delta = 2$ | 1 | +3.773 | 1 | 1.094 | 1 | 0.763 |
| standardised mean $\mu(h)$ | | | 0.279 | 0.936 | 1.336 | 1.326 |
| predmean $= m_y + s_y\,\mu(h)$ | | | 1063.4 | 1352.9 | 1529.4 | 1525.0 |
| predsd | | | 177.2 | 238.8 | 372.8 | 562.4 |

**Table 7:** The decomposition (6.5) for Belgium ARI. For comparison, the prior standard deviation $s_y\sqrt{\sigma^2 + V_e}$ is 861.6 and the last observed value is 1071.3.

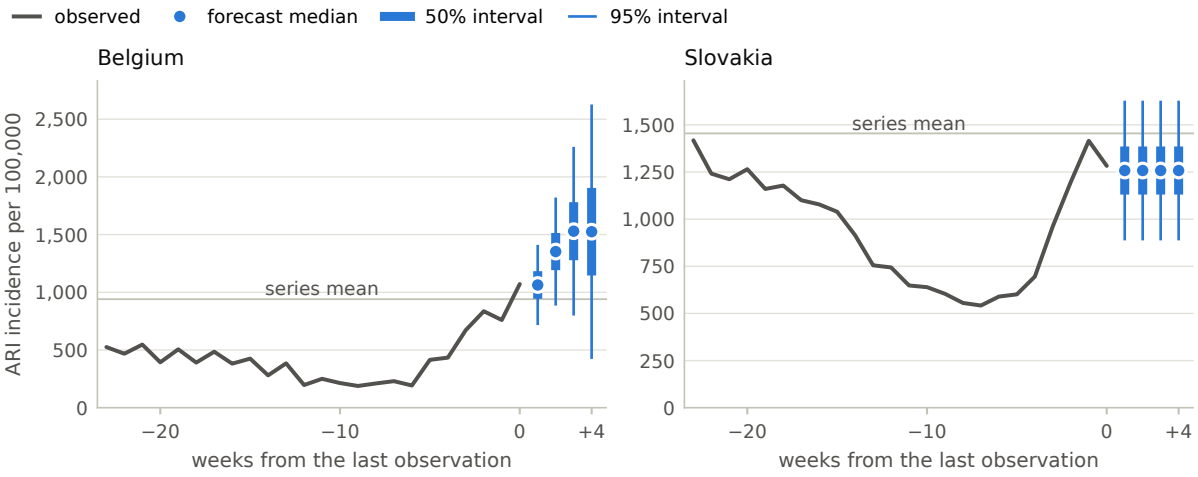



**Figure 5:** Forecasts of the round with origin date 14 October 2026 for two ARI series, each with the blanked week as its only gap. In Belgium the fit keeps the time gap ($\varphi_\Delta = 4.1 \times 10^{-4}$) and the four horizons differ. In Slovakia the fit switches it off and the four forecasts are the same number with the same interval.

# 7 Where the port follows R, and where it does not

The port reproduces the following behaviour of GPEDM on purpose. All but the last are marked `NOTE` in the source.

- The likelihood gradient of the $\varphi_k$ carries the factor one half (4.5).
- The prior gradient of a fixed parameter stays in $\boldsymbol{g}$ (Section 4.3, vi).
- `makelags(vtimestep=True)` returns its rows grouped by population, in order of first appearance. They line up with the data only if the data are sorted the same way.
- The default `Tdiff_fore` is computed from the first population and reused for the others.
- The exclusion radius of leave-one-out and leave-time-out counts rows and positions, not time.
- In sequential prediction the first time point is reported with $\sigma^2 + V_e$ where the other rows carry a standard deviation, so that entry is a variance.
- The lower bound of $\rho$ is 0 when the start value $\rho_0$ is transformed and $10^{-4}$ in the likelihood, so with several populations the optimiser starts from $10^{-4} + (1 - 10^{-4})\,\rho_0$, not from $\rho_0$.

The port differs from R in two respects.

- **No complete rows.** R fits zero rows without an error and its `predict` returns missing values. `gpedm_py` raises an error that names the constant columns.
- **Rounding.** R standardises the training data with `scale()` and new inputs with `mean()` and `sd()`, and links another linear-algebra library. Results differ in the last digits, amplified by the condition number of $\boldsymbol{\Sigma}$: at most $1.1 \times 10^{-6}$ relative in the 36 series compared, for a matrix with condition number $2.7 \times 10^{6}$.

Not ported: data augmentation (`augment`, `augdata`), `linprior`, `rhomatrix`, lags generated inside `fitGP` through its `E` and `tau` arguments, `returnGPgrad`, the per-population $R^2$ tables, and the fisheries and iterated-forecast functions.

# A Symbols and their names in the code

| symbol | in `gpedm_py.py` | where |
|---|---|---|
| $y_i$, $t_i$ | the `y` and `time` columns of `data` | `makelags`, `fitGP` |
| $O$ | `nm` | `makelags` |
| $\ell_{k-1}(i)$, $\ell_k(i)$ | `prev`, `new` | `makelags` |
| $\Delta_{ik}$ | columns `Tdiff_1`, ..., `Tdiff_E` | output of `makelags` |
| $h_1, \ldots, h_H$ | `Tdiff_fore` | `makelags` |
| $m_y$, $s_y$, $m_k$, $s_k$ | `ymeans`, `ysds`, `xmeans`, `xsds` | `fit.scaling` |
| $\boldsymbol{Y}$, $\boldsymbol{X}$ | `Y`, `X` | `fit.inputs` |
| $\boldsymbol{\theta}$ | `pars`: `phi1`, ..., `ve`, `sigma2`, `rho` | `fit.pars` |
| $\boldsymbol{p}$ | `parst` | `fit.parst` |
| $\boldsymbol{C}$, $\boldsymbol{\Sigma}$, $\boldsymbol{\Sigma}^{-1}$ | `Cd`, `Sigma`, `iKVs` | `fit.covm` |
| $\boldsymbol{R}$, $\boldsymbol{a}$, $\boldsymbol{Q}$, $\boldsymbol{D}^{(k)}$ | `L`, `a`, `Q`, `D[k]` | `getlikegrad` |
| $\lambda$, $m$ | `lam_phi`, `modeprior` | `getpriors` |
| $F$, $\boldsymbol{g}$ | `nllpost`, `grad` | `getlikegrad`, `fit` |
| $\boldsymbol{\delta}$, $q$, $t$ | `delta`, `df`, `it` | `fmingrad_Rprop` |
| $\boldsymbol{c}_*$ | `Cs` | `predict` |
| $m_y + s_y\mu$, $s_y\sqrt{v}$, $s_y\sqrt{v+V_e}$ | `predmean`, `predfsd`, `predsd` | result tables |
| *the forecast from 21 October 2026* (Section 6.1) | | |
| $c$ | the number in the transform name `log:`$c$ | `gpedm_direct.log_transform` |
| $z_i$ | `z` | `direct_forecast` |
| $k$, $H = k + h$ | `weeks_missing`, `horizons` | `forecast`, `direct_forecast` |
| $\boldsymbol{x}_s^{(H)}$, $y_s^{(H)}$ | columns `L0`, ..., `L`$E-1$, `sea_sin`, `sea_cos`; `y` | `direct_design` |
| $z_n + \mu_H$, $\sigma_H$ | `Forecast.mean`, `Forecast.sd` | `gpedm_direct.forecast` |
| $\hat{x}_q$ | `Forecast.quantiles` | `gpedm_direct.quantiles` |

# References


[1] S. B. Munch and T. L. Rogers. *GPEDM: Gaussian Process regression for Empirical Dynamic Modeling*. R package, version 0.0.0.9010. https://tanyalrogers.github.io/GPEDM

[2] S. B. Munch, V. Poynor and J. L. Arriaza. Circumventing structural uncertainty: a Bayesian perspective on nonlinear forecasting for ecology. *Ecological Complexity* 32 (2017), 134–143.

[3] B. Johnson and S. B. Munch. An empirical dynamic modeling framework for missing or irregular samples. *Ecological Modelling* 468 (2022).

[4] C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[5] M. Riedmiller and H. Braun. A direct adaptive method for faster backpropagation learning: the RPROP algorithm. *IEEE International Conference on Neural Networks* (1993), 586–591.

[6] C. Igel and M. Hüsken. Empirical evaluation of the improved Rprop learning algorithms. *Neurocomputing* 50 (2003), 105–123.